

Titanic Dataset: Detailed Analysis Report

1. Introduction

This report contains a comprehensive analysis of the Titanic dataset using Python and popular data science libraries like Pandas, NumPy, Matplotlib, and Seaborn. The objective is to explore the dataset, uncover meaningful insights, and understand the factors influencing passenger survival.

2. Code Section

```
>>> Code:
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

3. Code Section

```
>>> Code:
# Load the dataset

path = ("C:/Users/satish/Downloads/intern_task-5/train.csv")

df = pd.read_csv(path)

print(df)
```

>>> Output:

PassengerId	Survived	Pclass	\
0	1	0	3
1	2	1	1
2	3	1	3
3	4	1	1
4	5	0	3
..	...	...	...
886	887	0	2
887	888	1	1
888	889	0	3
889	890	1	1
890	891	0	3

	Name	Sex	Age	SibSp	\
0	Braund, Mr. Owen Harris	male	22.0	1	
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	
2	Heikkinen, Miss. Laina	female	26.0	0	
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	
4	Allen, Mr. William Henry	male	35.0	0	
..	...	...	...	...	
886	Montvila, Rev. Juozas	male	27.0	0	

Titanic Dataset: Detailed Analysis Report

887	Graham, Miss. Margaret Edith	female	19.0	0
888	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1
889	Behr, Mr. Karl Howell	male	26.0	0
890	Dooley, Mr. Patrick	male	32.0	0

	Parch	Ticket	Fare	Cabin	Embarked
0	0	A/5 21171	7.2500	NaN	S
1	0	PC 17599	71.2833	C85	C
2	0	STON/O2. 3101282	7.9250	NaN	S
3	0	113803	53.1000	C123	S
4	0	373450	8.0500	NaN	S
..	...	...	...	...	...
886	0	211536	13.0000	NaN	S
887	0	112053	30.0000	B42	S
888	2	W./C. 6607	23.4500	NaN	S
889	0	111369	30.0000	C148	C
890	0	370376	7.7500	NaN	Q

[891 rows x 12 columns]

4. Code Section

>>> Code:

```
# Basic Exploration of dataset
```

```
print("\nData Information:")
print(df.info())
print("***100)
```

```
print("\nData Description:")
print(df.describe())
print("***100)
```

```
print("\nValue Counts For 'Survived':")
print(df["Survived"].value_counts())
print("Here '0' refers Died")
print("Here '1' refers Alive")
```

>>> Output:

Data Information:

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 891 entries, 0 to 890

Data columns (total 12 columns):

#	Column	Non-Null Count	Dtype
0	PassengerId	891 non-null	int64
1	Survived	891 non-null	int64
2	Pclass	891 non-null	int64
3	Name	891 non-null	object

Titanic Dataset: Detailed Analysis Report

```
4  Sex      891 non-null  object
5  Age      714 non-null  float64
6  SibSp    891 non-null  int64
7  Parch    891 non-null  int64
8  Ticket   891 non-null  object
9  Fare     891 non-null  float64
10 Cabin    204 non-null  object
11 Embarked 889 non-null  object
```

dtypes: float64(2), int64(5), object(5)

memory usage: 83.7+ KB

None

```
*****
**
```

Data Description:

	PassengerId	Survived	Pclass	Age	SibSp \
count	891.000000	891.000000	891.000000	714.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008
std	257.353842	0.486592	0.836071	14.526497	1.102743
min	1.000000	0.000000	1.000000	0.420000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000
50%	446.000000	0.000000	3.000000	28.000000	0.000000
75%	668.500000	1.000000	3.000000	38.000000	1.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000

	Parch	Fare
count	891.000000	891.000000
mean	0.381594	32.204208
std	0.806057	49.693429
min	0.000000	0.000000
25%	0.000000	7.910400
50%	0.000000	14.454200
75%	0.000000	31.000000
max	6.000000	512.329200

```
*****
**
```

Value Counts For 'Survived':

Survived

0 549

1 342

Name: count, dtype: int64

Here '0' refers Died

Here '1' refers Alive

5. Code Section

>>> Code:

Checking the Missing Values

Titanic Dataset: Detailed Analysis Report

```
print("\nMissing Values Before Replacement")
print(df.isna().sum())
```

>>> Output:

Missing Values Before Replacement

```
PassengerId      0
Survived          0
Pclass           0
Name             0
Sex              0
Age             177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin           687
Embarked         2
dtype: int64
```

6. Code Section

>>> Code:

```
# Here Age, Cabin and Embarked has missing values
```

7. Code Section

>>> Code:

```
# Replace Missing Values
# For Age replace with median
```

```
df["Age"] = df["Age"].fillna(df["Age"].median())
```

8. Code Section

>>> Code:

```
# For Embarked replace with mode
```

```
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])
```

9. Code Section

>>> Code:

```
# For Cabin replace with
```

```
df["Cabin"] = df["Cabin"].fillna("Unknown")
```

Titanic Dataset: Detailed Analysis Report

10. Code Section

```
>>> Code:
# Checking missing values for filling the values
```

```
df.isna().sum()
```

```
>>> Output:
```

```
PassengerId    0
Survived        0
Pclass          0
Name            0
Sex             0
Age            0
SibSp          0
Parch          0
Ticket         0
Fare           0
Cabin          0
Embarked        0
dtype: int64
```

11. Code Section

```
>>> Code:
```

```
df.dtypes
```

```
>>> Output:
```

```
PassengerId    int64
Survived        int64
Pclass          int64
Name            object
Sex             object
Age            float64
SibSp          int64
Parch          int64
Ticket         object
Fare           float64
Cabin          object
Embarked        object
dtype: object
```

12. Code Section

```
>>> Code:
```

```
# Data Visualization using Matplotlib and seaborn
```

```
# Histogram
```

Titanic Dataset: Detailed Analysis Report

```
df.hist(figsize = (12, 10), color = 'skyblue', edgecolor = 'blue')
plt.suptitle('Histograms of Titanic Features', fontsize = 16)
plt.show()
```

>>> Output:
[Visual Output]

13. Code Section

```
>>> Code:

# Observation 1: PassengerId distribution
print("PassengerId is uniformly distributed.")
print("""120)

# Observation 2: Survived distribution
survival_counts = df['Survived'].value_counts()
print("\nSurvival Counts:\n", survival_counts)
if survival_counts[0] > survival_counts[1]:
    print("More passengers did NOT survive.")
else:
    print("More passengers survived.")
print("""120)

# Observation 3: Pclass distribution
pclass_counts = df['Pclass'].value_counts()
print("\nPassenger Class Counts:\n", pclass_counts)
print(f"Most passengers belong to class {pclass_counts.idxmax()}.")
print("""120)

# Observation 4: Age distribution
print("\nAge Statistics:")
print(df['Age'].describe())
print("Most passengers are between 20 and 40 years old.")
print("""120)

# Observation 5: SibSp distribution
sibsp_counts = df['SibSp'].value_counts()
print("\nSibSp (Siblings/Spouses aboard) Counts:\n", sibsp_counts)
print("Most passengers had no siblings or spouses aboard.")
print("""120)

# Observation 6: Parch distribution
parch_counts = df['Parch'].value_counts()
print("\nParch (Parents/Children aboard) Counts:\n", parch_counts)
print("Most passengers had no parents or children aboard.")
print("""120)

# Observation 3: Fare distribution
```

Titanic Dataset: Detailed Analysis Report

```
print("Fare distribution is right-skewed, with most fares under $100 and a few outliers with very high fares.")
```

>>> Output:

PassengerId is uniformly distributed.

```
*****
*****
```

Survival Counts:

Survived

0 549

1 342

Name: count, dtype: int64

More passengers did NOT survive.

```
*****
*****
```

Passenger Class Counts:

Pclass

3 491

1 216

2 184

Name: count, dtype: int64

Most passengers belong to class 3.

```
*****
*****
```

Age Statistics:

count 891.000000

mean 29.361582

std 13.019697

min 0.420000

25% 22.000000

50% 28.000000

75% 35.000000

max 80.000000

Name: Age, dtype: float64

Most passengers are between 20 and 40 years old.

```
*****
*****
```

SibSp (Siblings/Spouses aboard) Counts:

SibSp

0 608

1 209

2 28

4 18

3 16

8 7

5 5

Titanic Dataset: Detailed Analysis Report

Name: count, dtype: int64

Most passengers had no siblings or spouses aboard.

```
*****
*****
```

Parch (Parents/Children aboard) Counts:

```
Parch
0      678
1      118
2       80
5        5
3        5
4        4
6         1
```

Name: count, dtype: int64

Most passengers had no parents or children aboard.

```
*****
*****
```

Fare distribution is right-skewed, with most fares under \$100 and a few outliers with very high fares.

14. Code Section

>>> Code:

```
# Boxplots
```

```
plt.figure(figsize = (8,6))
sns.boxplot(x = 'Survived', y = 'Age', data = df)
plt.title("Boxplot of Age by 'Survived'")
plt.show()
print("Here '0' refers Died")
print("Here '1' refers Alive")
```

>>> Output:

```
[Visual Output]Here '0' refers Died
Here '1' refers Alive
```

15. Code Section

>>> Code:

```
# Observations
```

```
died_age_stats = df[df['Survived'] == 0]['Age'].describe()
survived_age_stats = df[df['Survived'] == 1]['Age'].describe()
```

```
print("Age Statistics for passengers who died (Survived=0):")
print(died_age_stats)
print("""*120)
```

```
print("\nAge Statistics for passengers who survived (Survived=1):")
```


Titanic Dataset: Detailed Analysis Report

```
print(survived_age_stats)
print("***120)

# Example insights:
if died_age_stats['mean'] > survived_age_stats['mean']:
    print("\nObservation: On average, passengers who died were older than those who survived.")
else:
    print("\nObservation: On average, passengers who survived were older than those who died.")

if died_age_stats['25%'] > survived_age_stats['25%']:
    print("Younger passengers were more likely to survive.")
else:
    print("Survivors include many young passengers.")
```

>>> Output:

Age Statistics for passengers who died (Survived=0):

count	549.000000
mean	30.028233
std	12.499986
min	1.000000
25%	23.000000
50%	28.000000
75%	35.000000
max	74.000000

Name: Age, dtype: float64

Age Statistics for passengers who survived (Survived=1):

count	342.000000
mean	28.291433
std	13.764425
min	0.420000
25%	21.000000
50%	28.000000
75%	35.000000
max	80.000000

Name: Age, dtype: float64

Observation: On average, passengers who died were older than those who survived.
Younger passengers were more likely to survive.

16. Code Section

>>> Code:

```
# ScatterPlot
```

Titanic Dataset: Detailed Analysis Report

```
plt.figure(figsize = (8,6))
sns.scatterplot(x = 'Age', y = 'Fare', hue = 'Survived', data = df)
plt.title("Scatterplot of Age 'V/s' Fare by Survival")
plt.show()
print("Here '0' refers Died")
print("Here '1' refers Alive")
```

```
>>> Output:
[Visual Output]Here '0' refers Died
Here '1' refers Alive
```

17. Code Section

```
>>> Code:
# Observations
print("Observations:")
print("1. Majority of passengers paid fare below 100.")
print("2. Younger passengers (age < 15) have a higher survival rate (more orange dots).")
print("3. High fare passengers (fare > 200) show more survival cases (orange).")
print("4. Elderly passengers (age > 60) have mixed survival outcomes.")
print("5. Survival seems more frequent in passengers with moderate age and higher fare.")
```

```
# Optional: You can break this down into more analytical code if needed.
```

```
>>> Output:
Observations:
1. Majority of passengers paid fare below 100.
2. Younger passengers (age < 15) have a higher survival rate (more orange dots).
3. High fare passengers (fare > 200) show more survival cases (orange).
4. Elderly passengers (age > 60) have mixed survival outcomes.
5. Survival seems more frequent in passengers with moderate age and higher fare.
```

18. Code Section

```
>>> Code:
# Pairplot

sns.pairplot(df[['Survived', 'Age', 'Fare', 'Pclass']], hue = 'Survived')
plt.suptitle("Pairplot of Selected Features", y = 1.02)
plt.show()
```

```
>>> Output:
[Visual Output]
```

19. Code Section

```
>>> Code:
# Observations based on the pairplot
```

Titanic Dataset: Detailed Analysis Report

```
observations = """
```

1. Fare vs Pclass: Passengers in lower classes (e.g., Pclass 3) generally paid lower fares, while higher class passengers (Pclass 1) paid more.
2. Fare vs Survived: Survived passengers tend to have paid higher fares, indicating a potential survival advantage for wealthier passengers.
3. Age Distribution: There is no clear age boundary for survival, but many survivors are observed in younger age ranges.
4. Pclass vs Survived: Higher survival rate is seen in Pclass 1 compared to Pclass 3, suggesting class significantly impacted survival.
5. Age vs Fare: No strong correlation, but most passengers who paid high fares were also from a smaller age group (middle-aged).

```
"""
```

```
print("Observations from the pairplot:")  
print(observations)
```

>>> Output:

Observations from the pairplot:

1. Fare vs Pclass: Passengers in lower classes (e.g., Pclass 3) generally paid lower fares, while higher class passengers (Pclass 1) paid more.
2. Fare vs Survived: Survived passengers tend to have paid higher fares, indicating a potential survival advantage for wealthier passengers.
3. Age Distribution: There is no clear age boundary for survival, but many survivors are observed in younger age ranges.
4. Pclass vs Survived: Higher survival rate is seen in Pclass 1 compared to Pclass 3, suggesting class significantly impacted survival.
5. Age vs Fare: No strong correlation, but most passengers who paid high fares were also from a smaller age group (middle-aged).

20. Code Section

>>> Code:

```
# Heatmap (Correlation)  
plt.figure(figsize=(8,6))  
sns.heatmap(df.select_dtypes(include=['float64', 'int64']).corr(), annot=True, cmap='coolwarm',  
linewidths=0.5)  
plt.title("Heatmap of Correlation")  
plt.show()
```

>>> Output:

[Visual Output]

21. Code Section

>>> Code:

```
#Observations  
print("\nKey Observations:")  
# Top positively correlated with 'Survived'
```

Titanic Dataset: Detailed Analysis Report

```
print("1. Survived is moderately positively correlated with Fare (0.26).")
print("2. Survived has weak positive correlation with Parch (0.082).")
# Top negatively correlated with 'Survived'
print("3. Survived is moderately negatively correlated with Pclass (-0.34).")
print("4. Survived has weak negative correlation with Age (-0.065).")
# Other noticeable correlations
print("5. Pclass and Fare have a strong negative correlation (-0.55).")
print("6. SibSp and Parch show a moderate positive correlation (0.41).")
```

>>> Output:

Key Observations:

1. Survived is moderately positively correlated with Fare (0.26).
2. Survived has weak positive correlation with Parch (0.082).
3. Survived is moderately negatively correlated with Pclass (-0.34).
4. Survived has weak negative correlation with Age (-0.065).
5. Pclass and Fare have a strong negative correlation (-0.55).
6. SibSp and Parch show a moderate positive correlation (0.41).

22. Code Section

>>> Code:

```
# Observations (printed directly here)
observations = """
Observations:
- Survival Rate: More people did not survive (0) compared to survivors (1).
- Missing Values: After replacement, no missing values remain.
- Age: Younger passengers had slightly higher survival rates.
- Fare: Higher fare seems associated with higher survival.
- Pclass: Lower class (higher number) had lower survival rates.
- Correlations: Fare and Pclass negatively correlated; Survived positively correlated with Fare.
"""

print(observations)

# Summary
summary = """
Summary of Findings:
- Younger and wealthier passengers had higher survival probabilities.
- First class passengers survived more compared to second and third classes.
- Age and Fare show some positive correlation with survival.
- Missing values have been handled properly.
- Fare is skewed; could consider transformations.
"""

print(summary)
```

>>> Output:

Observations:

- Survival Rate: More people did not survive (0) compared to survivors (1).
- Missing Values: After replacement, no missing values remain.
- Age: Younger passengers had slightly higher survival rates.

Titanic Dataset: Detailed Analysis Report

- Fare: Higher fare seems associated with higher survival.
- Pclass: Lower class (higher number) had lower survival rates.
- Correlations: Fare and Pclass negatively correlated; Survived positively correlated with Fare.

Summary of Findings:

- Younger and wealthier passengers had higher survival probabilities.
- First class passengers survived more compared to second and third classes.
- Age and Fare show some positive correlation with survival.
- Missing values have been handled properly.
- Fare is skewed; could consider transformations.

23. Code Section

>>> Code: